*A PROJECT REPORT ON*

# GRAPHRAG-NET: A PROTOTYPE FOR HYBRID RETRIEVAL AND EVIDENCE-LINKED CYBER THREAT INTELLIGENCE ANALYSIS

*Submitted in partial fulfillment of the requirements for the degree of*

**M.Tech. (Integrated) in Software Engineering**

*by*

**[AUTHOR NAME] (Reg. No. [REGISTRATION NUMBER])**

**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**

**VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI**

[SUBMISSION MONTH AND YEAR]

## TITLE PAGE

# GRAPHRAG-NET: A PROTOTYPE FOR HYBRID RETRIEVAL AND EVIDENCE-LINKED CYBER THREAT INTELLIGENCE ANALYSIS

*Submitted in partial fulfillment of the requirements for the degree of M.Tech. (Integrated) in Software Engineering*

**by**

**[AUTHOR NAME] (Reg. No. [REGISTRATION NUMBER])**

Under the guidance of
[GUIDE NAME AND DESIGNATION]

**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**
**VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI**
**[SUBMISSION MONTH AND YEAR]**

# DECLARATION BY THE CANDIDATE

I, [AUTHOR NAME], declare that the work presented in this project report, entitled "GraphRAG-Net: A Prototype for Hybrid Retrieval and Evidence-Linked Cyber Threat Intelligence Analysis," is my own work carried out under the supervision of [GUIDE NAME], subject to the source acknowledgements and limitations stated in the report.

I further declare that this report has not been submitted, in whole or in part, for another degree or diploma. This declaration is a fillable template field and must be reviewed and signed by the candidate before submission.

Place: [PLACE]
Date: [DATE]

Signature of the Candidate: _____

# CERTIFICATE

This page is reserved for completion by the project guide and the institution. The undersigned should verify the report and confirm whether it meets applicable university requirements before signing.

Report title: GraphRAG-Net: A Prototype for Hybrid Retrieval and Evidence-Linked Cyber Threat Intelligence Analysis

Guide name and designation: [TO BE COMPLETED]

Signature: _____

Date: [DATE]

Examiner 1: _____

Examiner 2: _____

# ABSTRACT

Cyber threat intelligence (CTI) is distributed across behavior taxonomies, vulnerability catalogues, weakness classifications, exploit-pattern libraries, and advisory text. Analysts must connect these heterogeneous records while preserving the evidence behind each conclusion. This report presents GraphRAG-Net, a prototype that integrates structured CTI sources into a typed knowledge graph and combines lexical retrieval, graph-neighborhood representations, reciprocal-rank fusion, evidence ranking, and grounded response generation. The implementation uses MITRE ATT&CK/STIX, CVE, CWE, CAPEC, and advisory-related artifacts represented in the project repository. The processed graph inputs comprise 6,012 unique entity identifiers and 27,352 edge records across the files consumed by the retrieval pipeline; edge records are not deduplicated and therefore are not equivalent to unique facts. A heterogeneous GraphSAGE encoder is trained with a self-supervised corrupted-destination objective, while hybrid retrieval joins BM25/TF-IDF rankings with GNN cosine similarity and attaches graph context for downstream reasoning.

The project also implements confidence features based on retrieval strength, cross-channel agreement, and answer grounding. However, inspection of the evaluation code shows that the 150-example calibration dataset derives its correctness labels from two of the same agreement features supplied to the calibrator. The saved AUROC, Brier, and expected calibration error values are therefore circular development diagnostics, not evidence of calibrated probabilities on independently judged answers. Likewise, the link-prediction evaluation script does not hold out test edges and does not implement a valid filtered candidate-ranking benchmark. Accordingly, this work reports implementation scope and data inventory, not validated gains over existing methods. The contribution is an inspectable integration prototype and a concrete evaluation protocol for future validation with independently annotated analyst queries, leakage-safe graph splits, and expert review.



**Keywords:** cyber threat intelligence; knowledge graph; GraphRAG; heterogeneous GraphSAGE; hybrid retrieval; evidence provenance; confidence calibration

# ACKNOWLEDGEMENT

[Complete this section with acknowledgements to the project guide, collaborators, institution, and other contributors. Names and contributions have not been inferred.]

# CONTENTS

Declaration by the Candidate ..... 3
Certificate ..... 4
Abstract ..... 5
Acknowledgement ..... 6
List of Figures ..... 8
List of Tables ..... 8
List of Abbreviations ..... 8
1. Introduction ..... 9
2. Literature Survey ..... 11
3. System Design and Methodology ..... 14
4. Implementation ..... 18
5. Evaluation and Results ..... 20
6. Discussion ..... 23
7. Conclusion and Future Work ..... 25
Appendix A. Reproducibility and Evaluation Checklist ..... 26
References ..... 28

# LIST OF FIGURES

Figure 1. GraphRAG-Net architecture and evidence flow.....14

# LIST OF TABLES

Table 1. Processed graph inventory.....15

Table 2. Comparison with representative literature.....13

Table 3. Development calibration diagnostics reported in project artifacts.....20

Table 4. Implemented evaluation status and required validation.....21

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| API | Application programming interface |
| ATT&CK | Adversarial Tactics, Techniques, and Common Knowledge |
| AUROC | Area under the receiver operating characteristic curve |
| BM25 | Best Matching 25 ranking function |
| CAPEC | Common Attack Pattern Enumeration and Classification |
| CISA KEV | Cybersecurity and Infrastructure Security Agency Known Exploited Vulnerabilities |
| CTI | Cyber threat intelligence |
| CVE | Common Vulnerabilities and Exposures |
| CWE | Common Weakness Enumeration |
| ECE | Expected calibration error |
| GNN | Graph neural network |
| KG | Knowledge graph |
| LLM | Large language model |
| RAG | Retrieval-augmented generation |
| RRF | Reciprocal rank fusion |
| STIX | Structured Threat Information Expression |
| TF-IDF | Term frequency-inverse document frequency |

# 1. INTRODUCTION

## 1.1 Context and motivation

Security analysts routinely correlate observations that are represented in different formats and maintained by different organizations. ATT&CK describes adversary tactics and techniques; CVE records identify disclosed vulnerabilities; CWE organizes software weaknesses; CAPEC describes attack patterns; and advisories add product-specific context. MITRE distributes ATT&CK as structured STIX data to support machine-readable workflows (MITRE, n.d.). These resources are valuable individually, but operational questions frequently require traversing between them: which weakness classes are associated with a vulnerability, which attack patterns exploit those weaknesses, which techniques are used by a threat group, and which mitigations relate to those techniques.

Knowledge graphs represent entities and typed relations explicitly, making multi-hop links available to retrieval and explanation. Retrieval-augmented generation can ground language-model output in external evidence rather than relying only on model parameters (Lewis et al., 2020). Graph-aware retrieval adds relational context to that process; recent work formalizes graph indexing, graph-guided retrieval, and graph-enhanced generation as distinct design stages (Peng et al., 2024; Hu et al., 2024). The engineering challenge is to combine heterogeneous, evolving CTI sources with useful retrieval while retaining provenance and communicating uncertainty.

## 1.2 Problem statement

The project addresses a systems problem: build a modular pipeline that links structured CTI entities and returns a query-focused evidence bundle with lexical and graph signals. The prototype also attempts to estimate answer correctness from evidence-strength features. This report evaluates what the repository demonstrably implements and whether its current artifacts support research claims. The distinction matters because functional integration, retrieval quality, answer factuality, and confidence calibration are separate properties; evidence for one does not establish the others.

## 1.3 Objectives

- Integrate selected ATT&CK/STIX, CVE, CWE, CAPEC, and advisory-related data into a typed graph representation.

- Generate node features from entity names and descriptions, train a heterogeneous neighborhood encoder, and expose graph-aware candidate retrieval.
- Fuse lexical and graph candidate rankings, rank connected evidence, and construct structured prompts or a deterministic fallback answer.
- Inspect confidence-scoring artifacts and determine whether their metrics establish calibration on independent ground truth.
- Compare the system design with relevant CTI knowledge-graph, graph-learning, GraphRAG, retrieval-fusion, and calibration research.

## 1.4 Contributions and scope

The report contributes (i) a reproducible architectural description of the repository implementation, (ii) a processed-graph inventory derived from the exact files loaded by the pipeline, (iii) a source-code-based audit of the current evaluation claims, and (iv) a comparative research survey that separates implementation coverage from measured performance. It does not claim a novel graph-learning algorithm, validated analyst utility, superior retrieval, or statistically significant performance. No independent annotation study, blinded analyst assessment, leakage-safe test benchmark, or measured latency study was found in the inspected project artifacts.

# 2. LITERATURE SURVEY

## 2.1 Cyber threat intelligence knowledge graphs

Sarhan and Spruit (2021) present Open-CyKG, a CTI graph built from unstructured APT reports using open information extraction, cybersecurity named-entity recognition, and entity canonicalization. Their design foregrounds extraction from narrative reports and fusion into a canonical graph. GraphRAG-Net instead consumes a mix of structured taxonomies and vulnerability data, with a small LLM-extracted supplement, and emphasizes analyst-query retrieval across linked concepts. The two systems therefore address complementary parts of CTI knowledge integration: narrative extraction and canonicalization versus structured cross-source traversal. This project does not evaluate extraction quality against Open-CyKG.

Zhang et al. (2024) combine CTI knowledge with security-requirement data in an RCTI graph and propose edge propagation for heterogeneous link prediction. They report evaluation on three public datasets and manual review of proposed CTI-to-requirement links, describing a 97% connectivity rate for attack patterns and relevant security requirements. That result is specific to their graph, prediction task, and manual validation protocol. It cannot be compared numerically with GraphRAG-Net, whose stored link-prediction script lacks a held-out test set and whose graph does not include the same requirements ontology.

## 2.2 Graph learning and retrieval

GraphSAGE learns an inductive aggregation function over node features and local neighborhoods (Hamilton et al., 2017). The repository adapts the SAGE convolution operator through relation-specific heterogeneous convolutions, using TF-IDF/SVD node features and a corrupted-destination binary objective. This is a practical baseline design rather than a new GraphSAGE variant. A research comparison would require held-out links, relation-specific negative sampling, filtered ranking metrics, and comparison with simpler embedding and graph baselines.

Reciprocal rank fusion (RRF) combines rankings without requiring their raw scores to share a scale (Cormack et al., 2009). GraphRAG-Net applies RRF to a text list and a graph list, after deriving graph seeds from the top text candidates. This makes RRF an appropriate engineering component but does not independently establish retrieval effectiveness. In particular, the structural channel is conditioned on lexical retrieval and can miss graph-relevant concepts that text retrieval fails to seed.

## 2.3 Retrieval-augmented and graph-aware generation

Lewis et al. (2020) introduced a retrieval-augmented language-model framework for knowledge-intensive NLP, showing that a retriever-generator combination can use non-parametric evidence during generation. Graph-aware approaches extend retrieval to linked entities or textual subgraphs. Hu et al. (2024) describe graph-context retrieval and generation for networked documents, while Peng et al. (2024) survey the broader GraphRAG pipeline and its open evaluation challenges. GraphRAG-Net follows this general pattern by passing ranked entities and relation triplets into a constrained answer prompt. Its deterministic fallback improves availability but uses templated statements and must be separately audited for factual support; pipeline completion is not a substitute for answer-quality measurement.

## 2.4 Confidence and calibration

A confidence score is calibrated when predictions assigned probability p are correct at approximately frequency p across relevant cases. Guo et al. (2017) study post-hoc calibration and demonstrate temperature scaling for neural classifiers. Naeini et al. (2015) describe Bayesian binning into quantiles and discuss probability calibration with real and simulated data. ECE and Brier score summarize different aspects of probabilistic predictions, but they require meaningful labels and careful evaluation. In GraphRAG-Net, the labels are generated from the agreement and evidence-cosine features being used as predictors; high apparent discrimination can arise by construction and should not be interpreted as real correctness prediction.

**Table 2. Comparison with representative literature**

| Work | Problem and method | Evaluation reported in source | Relation to this project |
|---|---|---|---|
| Sarhan & Spruit (2021), Open-CyKG | CTI graph from APT reports using OIE, cybersecurity NER, and canonicalization. | Framework and retrieval examples; not a shared benchmark with this project. | GraphRAG-Net uses mostly structured taxonomies and vulnerability records; it does not match Open-CyKG extraction or evaluation. |
| Zhang et al. (2024), RCTI/EGNN | Heterogeneous CTI plus security-requirement KG; edge propagation for link prediction. | Three public datasets; paper reports 97% manual connectivity for inferred attack-pattern to security-requirement links. | Distinct graph schema and task; GraphRAG-Net currently has no valid held-out ranking score. |
| Hamilton et al. (2017), GraphSAGE | Inductive neighborhood aggregation from node attributes. | Node classification on citation, Reddit, and protein interaction settings. | GraphRAG-Net uses SAGEConv within a heterogeneous encoder; task/domain differ and adaptation is not an empirical win. |
| Cormack et al. (2009), RRF | Rank aggregation across information-retrieval systems. | TREC and LETOR experiments in the original study. | GraphRAG-Net uses RRF to combine channel rankings; no project-level retrieval benchmark is available. |
| Lewis et al. (2020), RAG | Retriever plus generator for knowledge-intensive NLP. | Open-domain QA benchmarks including Natural Questions, TriviaQA, and WebQuestions. | Conceptual grounding baseline; GraphRAG-Net focuses on typed CTI entities and graph paths. |
| Hu et al. (2024), GRAG | Retrieval and generation over textual graphs/subgraphs. | Graph reasoning benchmarks; multi-hop gains reported by the authors. | Related graph-aware generation design, but task/data and metrics are not directly comparable. |
| Guo et al. (2017); Naeini et al. (2015) | Post-hoc calibration and probabilistic calibration methods. | Classification studies with reference labels and reliability evaluation. | Motivates metrics and calibration; project proxy labels violate independent evaluation assumptions. |

Interpretation: the table compares research questions, system components, and evaluation designs. Published numerical results are retained only where they clarify a cited study's own protocol; they are not ranked against GraphRAG-Net because the datasets, targets, and metrics differ.

# 3. SYSTEM DESIGN AND METHODOLOGY

## 3.1 End-to-end architecture

The application is organized as an ingestion and graph-construction stage, node-feature generation and graph learning, dual-channel retrieval, evidence ranking, answer generation, and a confidence-scoring component. Figure 1 summarizes data flow as implemented. The confidence component consumes signals from retrieval, evidence agreement, and generated-response checks; because the current supervision is synthetic, its displayed probability is a prototype interface value rather than a validated probability of correctness.

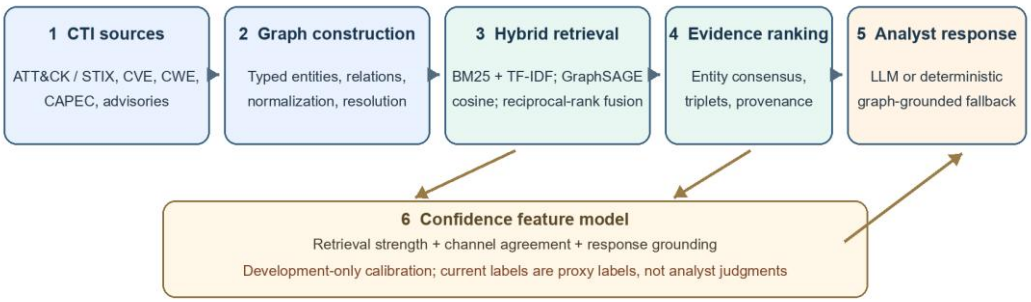



**Figure 1. GraphRAG-Net architecture and evidence flow. Confidence scoring is shown as an experimental subsystem; its current proxy labels do not support calibrated correctness claims.**

## 3.2 Data sources and graph schema

The repository includes downloaded or processed ATT&CK/STIX, CAPEC, CWE, CVE, CISA KEV, GitHub advisory, and LLM-extracted artifacts. The implemented graph builders map node labels to node types and edge labels to typed relations. Identifiers are used to connect the source records; entity-resolution artifacts and inferred-edge files are additional inputs. Because the project does not preserve a single manifest recording source release identifiers, retrieval dates, license metadata, and record-level provenance for every processed record, the counts below describe the current processed snapshot rather than a stable benchmark dataset.

**Table 1. Processed graph inventory**

| Loaded file group | Records present | Pipeline treatment / interpretation |
|---|---|---|
| nodes.json | 1,915 entities | ATT&CK-derived/general node store; de-duplicated by identifier when hybrid retrieval loads nodes. |
| capec_nodes.json | 1,828 entities | CAPEC node records. |
| cwe_nodes.json | 898 entities | CWE node records. |
| cve_nodes.json | 1,660 entities | CVE node records. |
| llm_extracted_nodes.json | 8 entities | Small supplemental extracted set; extraction review log is empty. |
| All node files | 6,012 unique identifiers | Unique IDs across the files loaded by hybrid retrieval; IDs, labels, and source fields may still need stronger validation. |
| All edge files | 27,352 edge records | Sum across edges.json (21,041), CAPEC (3,337), CWE (1,376), CVE (1,593), LLM-extracted (4), and inferred (1). Records are not deduplicated; count is not unique facts. |

The merged node labels include vulnerabilities, weaknesses, mitigations, software, techniques, attack patterns, threat groups, data components, campaigns, tactics, and two singleton metadata types. The edge labels include USES, EXPLOITS_WEAKNESS, MITIGATES, CHILD_OF, PART_OF_TACTIC, SUBTECHNIQUE_OF, CAN_PRECEDE, MAPS_TO_TECHNIQUE, and several lower-frequency relations. Different source files can encode overlapping content; a publishable dataset release should report exact release IDs and counts after identifier and duplicate-edge normalization.

## 3.3 Node features and heterogeneous encoder

For each entity type, the graph builder vectorizes node names and descriptions using TF-IDF, reduces the sparse vectors with truncated singular value decomposition, and pads or truncates to 128 dimensions. The encoder applies two heterogeneous convolution layers built from relation-specific SAGEConv modules, with a hidden dimension of 128 and a 64-dimensional output in the saved training configuration. To permit bidirectional message passing, the graph is transformed to include reverse edges; the self-supervised loss is computed over original edge types to avoid treating reverse copies as additional positive labels.

For an observed edge (u,r,v), the training objective compares the positive dot-product score between node embeddings with a negative edge made by replacing the destination with a randomly sampled node of the same destination type. Binary cross-entropy encourages positive scores above sampled negative scores. This objective is plausible for representation learning, but its current implementation uses all observed edges for training and does not create an independent test partition. Random corruptions can also sample a true but unobserved edge, so filtered negative sampling and type-appropriate evaluation are needed.

## 3.4 Hybrid retrieval and evidence ranking

The lexical channel combines BM25 and TF-IDF cosine similarity with weights 0.6 and 0.4 after BM25 score normalization. The top lexical entities seed a structural search over stored GNN embeddings. Structural retrieval averages the seed vectors and ranks all embedded nodes by cosine similarity. RRF adds reciprocal-rank contributions from both lists using k=60, and top candidates receive one-hop subgraph context. Because graph seeds are selected from the lexical top five, channel agreement is not an independent confirmation: it is a useful hybrid retrieval feature, but the dependency must be reflected in evaluation design.

The evidence-ranking stage merges query evidence and graph triplets, estimates composite support using channel consensus and source weights, and formats a compact evidence bundle. The response stage passes the bundle to a remote model when configured and uses a deterministic graph-based fallback when no model is available or a call fails. The prompt constrains unsupported identifiers, asks for attack-chain reconstruction and mitigations, and requires a provenance/confidence summary. The project should retain immutable source IDs, relation IDs, timestamps, and snippets in every returned claim so an analyst can independently inspect support.

## 3.5 Confidence feature design

The feature extractor groups nine numeric features into retrieval strength (top score, score margin, number of paths, average path length), cross-channel agreement (entity Jaccard, evidence cosine similarity, binary agreement), and response consistency/grounding (self-consistency score and unsupported-claim ratio). The calibrator trains logistic regression and gradient-boosting candidates with sigmoid calibration and selects using Brier score on a random 20% split. The label definition in the dataset builder is not expert or external ground truth: it thresholds a noisy combination of entity Jaccard and evidence cosine similarity, both also model inputs. This creates target leakage at the construct level even though the split is by example.

# 4. IMPLEMENTATION

## 4.1 Data ingestion and normalization

Download scripts retrieve public data into raw JSON artifacts, while source-specific parsers convert STIX, CWE, CVE, and CAPEC records into common node and edge structures. Graph assembly combines these structures for downstream retrieval and PyTorch Geometric conversion. This modular organization makes source adapters replaceable, but reproducibility requires version pinning: a branch-based ATT&CK URL, a live vulnerability API, and external advisory feeds can change over time. Each run should therefore save a frozen input manifest with URI, release/commit, retrieval timestamp, response checksum, parser version, and transformation settings.

## 4.2 Graph representation and training

The graph builder groups entities by their label and relations by (source type, relation, destination type), then writes a HeteroData object and identifier-to-index maps. Training is full-batch on CPU in the inspected implementation. The saved model and embeddings are consumed by the hybrid retriever. While GraphSAGE is designed around neighborhood sampling for scalability, this implementation materializes whole-graph features and edge indices; its current runtime and memory behavior should not be assumed to scale to large production graphs without measurement.

## 4.3 Analyst-facing interface

The React front end includes a query interface, graph visualization, evidence panel, reasoning report, and system statistics. The result payload exposes entities, relations, provenance categories, confidence subscores, and warning text. These interface components can support analyst review if each visual element is traceable to source records. Human factors evaluation is not present in the repository and remains necessary before claims about reduced analyst effort or improved decision quality.

## 4.4 Reproducibility details available in the repository

The GNN training script sets a PyTorch seed of 42 and specifies 100 epochs, hidden size 128, embedding size 64, and learning rate 0.005. Calibration uses an 80/20 stratified split with random_state=42 and three-fold sigmoid calibration inside candidate models. However, calibration dataset generation uses the Python random module without a recorded seed, and the saved dataset contains repeated variants of a small set of ten base query templates. The GraphSAGE run does not log software versions, hardware metadata, training curves beyond console loss, or a held-out graph split. These omissions limit exact replication of any performance claim.

# 5. EVALUATION AND RESULTS

## 5.1 Evidence available

The workspace contains source code, processed graph JSON files, a serialized heterogeneous graph and embeddings, a trained GNN model, a calibration model, a 150-line labeled calibration file, a reliability plot, and one reasoning-output example. The graph inventory is a reproducible count of the files consumed by the retriever, subject to duplicate records and source-snapshot caveats. No query-level relevance judgments, analyst-rated answer correctness labels, labeled link-prediction test set, or documented timing measurements were identified. Consequently, quantitative claims about retrieval precision/recall, factuality, latency, analyst utility, or generalization cannot be estimated from the current artifacts.

## 5.2 Calibration diagnostic values in project artifacts

The Stage 6 README records an 80/20 evaluation over 150 examples and gives the following values. They are reproduced here as artifact-reported diagnostics to document the implementation state, not as valid test performance. The data generator defines the binary target from Jaccard overlap and evidence cosine, then passes those signals to the model. In addition, query examples repeat variants of a small template set and the dataset generator is unseeded. Thus the metrics measure agreement with a synthetic rule on this generated sample, not correctness of analyst-facing answers.

**Table 3. Development calibration diagnostics reported in project artifacts**

| Feature/model group | ECE / Brier / AUROC | Interpretation |
|---|---|---|
| Retrieval features only | 0.4500 / 0.5079 / 0.2400 | README-reported proxy-label result; not correctness prediction. |
| Agreement features only | 0.1420 / 0.0854 / 0.9775 | Contains information used to construct proxy labels; apparent discrimination is circular. |
| Consistency features only | 0.4000 / 0.4000 / 0.5000 | README-reported proxy-label result. |
| Combined calibrated Gradient Boosting | 0.1285 / 0.0643 / 0.9850 | README-reported; not an independently labeled holdout result. |

The saved evaluation script computes ECE with ten fixed-width bins, Brier score, and AUROC on a 30-example split. ECE can be unstable for small samples and depends on the binning rule. A confidence model intended for security decision support should report uncertainty intervals, calibration curves with bin counts, class prevalence, decision-threshold behavior, subgroup and time-split results, and performance against independently adjudicated outcomes. The currently saved reliability diagram should be treated as an illustration of the synthetic-target experiment and should not be used as evidence of deployment calibration.

## 5.3 Link prediction evaluation audit

The link-prediction evaluation script trains each candidate encoder using the entire loaded graph and then evaluates on those same edge indices. Its relation-ranking procedure assigns similar or identical scores to relation candidates rather than evaluating candidate destinations for a held-out (head, relation, ?) query. The reported task names mention mean rank and Hits@k, but the implemented calculation does not establish those quantities for unseen links. No link-prediction score from this script is therefore reported as an empirical result in this paper.

## 5.4 Evaluation status and next experiment

**Table 4. Implemented evaluation status and required validation**

| Research question | Current artifact status | Valid present claim | Minimum next validation |
|---|---|---|---|
| Does hybrid retrieval find relevant entities? | No relevance labels or retrieval benchmark found. | The two retrieval channels and RRF are implemented. | Use held-out, expert-judged queries; report Recall@k, nDCG@k, MRR and channel ablations. |
| Does the GNN predict unseen typed edges? | Evaluation trains and scores on graph edges; ranking procedure is not valid filtered ranking. | A self-supervised encoder is trained on observed graph structure. | Split by relation and time; filtered MRR/Hits@k; compare TransE/DistMult, text-only and degree baselines. |
| Are generated answers correct and grounded? | No independent answer annotations or claim-level audit found. | Prompt and fallback include evidence entities and relations. | Blind expert annotation of correctness, completeness, support, and unsafe recommendations. |

| Research question | Current artifact status | Valid present claim | Minimum next validation |
|---|---|---|---|
| Does confidence estimate answer correctness? | Proxy target reuses predictor inputs; 150 generated examples. | A confidence-scoring implementation and development artifact exist. | Use independent human correctness labels, query-family/time holdouts, reliability metrics with intervals. |
| Does the system help analysts? | No user study or measured operational timing found. | A front end presents evidence and graph context. | Controlled analyst study against baseline workflow; task completion, error rate, time, and trust calibration. |

# 6. DISCUSSION

## 6.1 What the implementation establishes

The codebase establishes an end-to-end integration path across heterogeneous CTI records: parse and merge graph entities, construct typed neighborhoods, learn representations, retrieve text and structural candidates, rank evidence, and produce an answer payload. The graph inventory indicates substantial linked content in the current processed files. The modular design provides a foundation for experiments about hybrid retrieval and evidence-grounded reasoning. These are engineering contributions and useful research scaffolding; they are not yet evidence that the pipeline improves analyst decisions or generates more accurate answers than baselines.

## 6.2 Validity and reliability limitations

Construct validity is the central risk in the confidence experiment: a model trained to imitate a function of agreement features cannot demonstrate that those features predict answer correctness. Internal validity is also limited by the lack of an independent link test set, same-graph training and scoring, and weak candidate-rank semantics. External validity is unknown because the query set is generated from ten templates and data source versions are not frozen. Reproducibility is weakened by unseeded dataset construction and incomplete environment and source manifests. The source artifacts therefore support a faithful system description and evaluation audit, not a comparative effectiveness conclusion.

## 6.3 Security and operational considerations

CTI records can be incomplete, stale, or contradictory. A graph edge inferred by a model should remain distinguishable from a source-authored relation, with origin, timestamp, confidence, and review status preserved. The LLM prompt cannot by itself guarantee that every generated claim is supported; output claims should link back to evidence identifiers and be checked by analysts before operational action. External model calls may transfer sensitive query context and should be governed by organizational data-handling policy. Inferred links and low-confidence content should not automatically trigger blocking or remediation actions.

## 6.4 Comparison interpretation

The literature comparison shows that GraphRAG-Net combines known components in a domain-specific prototype: typed CTI graphs, neighborhood embeddings, lexical/structural retrieval, rank fusion, and grounded generation. Its nearest comparisons differ in dataset construction and objectives. Open-CyKG studies narrative extraction and graph canonicalization; RCTI/EGNN studies security-requirement linkage; GraphSAGE supplies an encoder family; GRAG and RAG study generation with retrieved graph/text evidence. Without a common benchmark, equivalent candidate sets, shared labels, and reproducible protocols, numerical ranking against these systems would be misleading. A defensible next paper should report separate experiments for graph completion, query retrieval, answer support, and calibration rather than a single overall score.

# 7. CONCLUSION AND FUTURE WORK

This report describes GraphRAG-Net as a prototype for connecting structured CTI sources and returning evidence-linked responses. The implementation combines source-specific ingestion, a heterogeneous graph, TF-IDF/SVD features, relation-aware GraphSAGE layers, hybrid lexical and graph retrieval, reciprocal-rank fusion, evidence ranking, and LLM or deterministic response generation. The loaded processed files contain 6,012 unique node identifiers and 27,352 edge records, with the caveat that edge duplicates remain and source release identifiers are not fully pinned.

The main result of the repository audit is methodological: current calibration and link-prediction artifacts do not support independent performance claims. Calibration labels are derived from input features; link evaluation has no held-out test edges and does not rank typed destination candidates correctly. Therefore, reported calibration values are documented as proxy diagnostics, and no link-prediction leaderboard result is claimed. No direct numerical comparison with published studies is possible on the available evidence.

Future work should first freeze and document data releases, deduplicate and validate graph records, and retain provenance at the relation level. It should then create leakage-safe temporal and relation-aware graph splits and compare standard knowledge-graph embedding, text-only, graph-only, and hybrid baselines. Query retrieval should be assessed on a publicly documented, expert-judged CTI query set. Generated claims and recommendations should receive blind analyst annotations. Confidence training must use labels independent from the feature formula and report reliability under distribution shift. Finally, a controlled analyst study should assess whether the interface improves task completion and error rates. Only after those experiments should the system be positioned as a validated publication contribution.

# APPENDIX A. REPRODUCIBILITY AND EVALUATION CHECKLIST

- Record immutable release tags or commit hashes for ATT&CK, CWE, CAPEC, CVE/NVD, CISA KEV, and advisory sources; save download timestamps and checksums.
- Publish the exact normalization, identifier-resolution, duplicate-edge, and inferred-edge policies; include source provenance for every edge.
- Seed every random generator and publish package versions, hardware, graph dimensions, training curves, and configuration files.
- Partition edges before training; ensure validation/test edges are excluded from message passing where the evaluation requires inductive or leakage-free prediction.
- Use type-constrained negatives and filtered ranking for link prediction; report MRR and Hits@1/3/10 with confidence intervals and relation-wise results.
- Construct query relevance judgments independently of the retrieval and confidence features; evaluate Recall@k, MRR, nDCG, and channel ablations.
- Obtain blinded human labels for answer correctness, claim support, completeness, and mitigation safety; split by query family and time.
- Train confidence calibration on human correctness labels; reserve a true test set and compare uncalibrated, sigmoid/Platt, isotonic, and binning methods.
- Report calibration curves, bin counts, ECE variants, Brier score, discrimination, and subgroup/time-shift behavior.
- Preserve model prompts, retrieved evidence IDs, generation model/version, decoding parameters, and all fallback behavior for audit.
- Complete candidate, guide, acknowledgement, date, signatures, and any institution-required page/format constraints before submission.

# APPENDIX B. CLAIMS SUPPORTED BY THE PRESENT REPOSITORY

| Claim class | Status | Evidence basis |
|---|---|---|
| Architecture and implemented components | Supported with code-level qualification | Pipeline scripts, model definitions, retrieval and response code, front-end components. |
| Current processed graph record counts | Supported as repository snapshot counts | Count of node and edge records in the input files consumed by the retrieval pipeline. |

| Claim class | Status | Evidence basis |
|---|---|---|
| Independent retrieval effectiveness | Not established | No independent query relevance labels or benchmark results found. |
| Independent link-prediction effectiveness | Not established | Training/evaluation split and rank calculation in the available script are invalid for held-out link ranking. |
| Calibration to answer correctness | Not established | Dataset target is constructed from predictor features; generated examples are not analyst ground truth. |
| Improved analyst outcomes | Not established | No controlled user study or operational outcome measurements found. |

# REFERENCES


Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (pp. 758–759). https://doi.org/10.1145/1571941.1572114

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In Proceedings of the 34th International Conference on Machine Learning (pp. 1321–1330). Proceedings of Machine Learning Research, 70. https://proceedings.mlr.press/v70/guo17a.html

Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In Advances in Neural Information Processing Systems (Vol. 30). https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html

Hu, Y., Lei, Z., Zhang, Z., Pan, B., Ling, C., & Zhao, L. (2024). GRAG: Graph retrieval-augmented generation. arXiv. https://arxiv.org/abs/2405.16506

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems (Vol. 33). https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

MITRE. (n.d.). ATT&CK data and tools. Retrieved September 29, 2026, from https://attack.mitre.org/resources/working-with-attack/

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. Proceedings of the AAAI Conference on Artificial Intelligence, 29(1), 2901–2907. https://doi.org/10.1609/aaai.v29i1.9602

Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., & Tang, S. (2024). Graph retrieval-augmented generation: A survey. arXiv. https://arxiv.org/abs/2408.08921

Sarhan, I., & Spruit, M. (2021). Open-CyKG: An open cyber threat intelligence knowledge graph. Knowledge-Based Systems, 233, 107524. https://doi.org/10.1016/j.knosys.2021.107524

Zhang, Y., Chen, J., Cheng, Z., Shen, X., Qin, J., Han, Y., & Lu, Y. (2024). Edge propagation for link prediction in requirement-cyber threat intelligence knowledge graph. Information Sciences, 653, 119770. https://doi.org/10.1016/j.ins.2023.119770